Cyphrpass

A self-sovereign perpetual identity and authentication layer for the Internet. Licensed under the Business Or Open source License (BOOL)

Introduction

Cyphrpass is the authentication layer for the Internet.

Cyphrpass verifies identities through public key authentication. Account state is represented by an updateable Merkle root. Unlike existing solutions, Cyphrpass is blockchain and cryptographic primitive agnostic, prioritizes key revocation and multiple devices, and introduces a new paradigm, **authenticated atomic action (AAA)**, which fundamentally rethinks user authentication architecture.

Cyphrpass uses a personal, user controlled, and decentralized ledger that provides a secure way to manage online identity. The ledger operates as a non-monetary blockchain with rules dictating state updates. Cyphrpass enables third parties to host user's data trustlessly, decomposing users from centralized services, enabling decentralized architectures and cross service verification.

Cyphrpass uses Coz, an open source, human readable cryptographic JSON messaging specification and cryptographic abstraction layer enabling cryptographic primitive independence and standardization. This document uses Coz labels: `pay`, `alg`, `now`, `tmb`, `typ`, `dig`, `msg`, `key`, `tag`, `pub`, `prv`,`rvk`, `can`, `cad`, `sig` and `czd`. Some Coz examples are truncated for brevity.

Key Cypherpass Features

- Universal, internet-wide authentication without the need for centralized login services.
- Password replacement.
- Email independent (no email password recovery).
- Domain and Certificate Authority independent.
- Service independent.
- Undermines walled gardens.
- Easy creation of anonymous accounts and subaccounts.
- Atomic action: each user action may be discreetly verifiable by all, removing the need for trusted services.
- A comment, bookmark, archival, backup, and "reaction/upvote/downvote" system for the whole web.

Part I: How Cyphrpass Works

Basic Principles

- **Keys and Wallet**- Each user wallet maintains cryptographic keys (private/public key pairs). Each Cyphrpass account maintains at least one key. Keys may be added or revoked over time and a Merkle root (MR) of these keys represents the current user state.
- Cyphrpass maintains two top level chains: an authentication Merkel root, the "Auth Ledger State" **(ALS)**, and an optional data Merkel root, "Data Ledger State" **(DLS)**, and a top level account MR, Account State **(AS)**.
- **Account Root (AR)** is the initial Merkel root for the account, which is a permanent cryptographic identifier for an account. AR is the initial AS.
- Wallets are kept in custody of the user. Cyphrpass is designed so that different devices may have different keys.
- **Transaction**- Users perform transactions using a key to sign a message that mutates the wallet state, such as adding or revoking a key, and by doing so the user's ALS is updated. Users store the history of transactions and publish transactions that update their state. Users and/or third parties keep a record of these transactions.
- **Login -** Instead of using passwords, to "sign up" or login, users sign a **proof of possession (pop)** action. On login, users provide a list of web addresses that may be queried for state updates. Since Cypherpass addresses are cryptographic digests, addresses themselves are globally namespaced. Further, users may enable automatic login. Further, instead of implementing a login system, AAA is suggested over an exclusive bearer token implementation. We also suggest services remove "sign up" and implement exclusively "login", reducing user friction.
- **Actions -** Like transactions, users may sign individual actions. Actions may be anything, like a comment, upvote, post, authorization to upload an image, or any other action by a user. Actions may be included in their own merkle tree separate from the authentication Merkle tree. (Level 4)
- **Authenticated Atomic Action (AAA)**- Instead of authenticating to a centralized login service, which is then trusted with a bearer token for user action, users may sign individual actions directly. Each atomic action may be authenticated by anyone, irrespective of centralized services. For example, instead of logging in to make a comment, a user signs a comment directly which is then verifiable by anyone. Centralized services that maintain user identity, like reddit user accounts, are no longer relevant and should be actively deprecated.
- Services themselves may be represented by a Cyphrpass account. Instead of services only tracking users, users may track service state, and services track user state. This is similar to double entry accounting, termed **Mutual State Synchronization (MSS)**. This allows users to track services outside of the certificate authority (CA) system, a long standing source of centralization.
- **Decentralized** - Cyphrpass's trustless design promotes decentralization. Services may query other services to fill in knowledge gaps or user data may be completely backed up by secondary services.
- **Recursive** - Cypherpass's datastructure is fully recursive.

- **Encapsulated** - Tree components are encapsulated inside of a digest. Encapsulated information may be hidden or disclosed as required promoting privacy. A nonce may also be used.
- **Internet Wide User State** - In addition to keys, users may store arbitrary data, like user images, inside the Merkel root tree. In this way, Cyphrpass may be used like Gravatar, providing user images, user information like address and contact information.
- **Time Aware** - Since user state changes over time, Cyphrpass is time aware. Messages signed after a key has been revoked are discarded.
- **Saved Content** - Services are constantly removing data, whether from censorship or users deleting accounts or posts, making finding information difficult. Universal identifiers help users save information locally without risk of services deleting information. Users may request historic posts from other users by content address.
- **Cryptographically Secure Journalism** - Nation states have incentive to change history to suit their whims. Without cryptographic hashing of archives, nation states may easily manipulate history. Cyphrpass removes the ability for changing history.
- **Designed assuming that data breaches are inevitable** - Keys are revocable.

Cyphrpass Feature Levels

Since services have varying authentication needs. Cyphrpass defines progressive feature levels enabling simple to complex implementations. Each level includes the previous level features. Services may implement the lower levels while ignoring the higher levels.

Level 1: One static key per service.

Level 2: One key with programmatic key replacement.

Level 3: Many keys (where each key equal authority).

Level 4: Arbitrary data (ASL, DLS, and AS).

Level 5: Weighted keys and timelock rules.

Level 6: Turing complete virtual machine.

Level 1 may be useful for dumb devices that don't have the complexity required for key updates, but can still perform cryptographic authentication. Level 1 and up is suitable for password replacement.

Level 2 is for slightly smarter dumb devices that can update keys. To limit complexity, still only one key is supported.

Level 3 is the critical feature level which enables various user devices to have separate keys. We recommend most services to implement minimally level 3. Level 3 defines the auth tree.

Level 4 increases the scope of Cyphrpass to include arbitrary user data, like actions. Since this data is hashed, it too is cryptographically integrity protected. The auth ledger state (ALS) and data ledger (DLS) are kept separate and are hashed into a single MR, the Account State (AS).

Level 5 allows users to set their own rules using a limited rule set: weighted keys and timelock rules.

Level 6 is full turing complete, allowing complete flexibility in implementation and design and is implemented us

Initializing and Updating State - Adding, Removing, and Revoking keys

Initially, the user creates one or more keys and calculates a Merkle root (MR) from the `tmb` of the keys. When only one key is present, the MR is calculated by taking the digest of the single key. (The MR is the digest of `tmb` so as not to conflict with the key's identifier.)

The initial MR is termed the **Account Root (AR)** which is a permanent cryptographic identifier for an account. As user state changes, services may use the original AR as an identifier or *any intermediate MR* as an identifier. Services may be unaware of the user's original state, but know a later state with a different MR.

Constructing and Calculating a AR for single keys

If a user account had only the following private key:


```
{
 "alg":"ES256",
 "iat":1623132000,
 "kid":"Zami's Majuscule Key.",
 "d":"bNstg4_H3m3SlROufwRSEgibLrBuRq9114OvdapcpVA",
 "tmb":"cLj8vsYtMBwYkzoFVZHBZo6SNL8wSdCIjCKAwXNuhOk",

"x":"2nTOaFVm2QLxmUO_SjgyscVHBtvHEfo2rq65MvgNRjORojq39Haq9rXNxvXxwba_Xj0F5vZibJR3isBdOWbo5g"
}
```


The AR is calculated as the first hash of the tmb, in this case: `RpMM4_lU6jCj3asZEtIFyYqPjC2L6mlucl7VGMvAuno`

State may be represented as a pair, AR:Current State. In this example, for account "RpMM4_..." state is TODO HASH "RpMM4_..." since there has not yet been an update.

Genesis Transaction

To explicitly create an account, a genesis transaction should be signed and published. The type must be `cyphrpass/account/create`, and include the `id`, which is the Merkle Root


```
{
 "pay": {
   "alg": "ES256",
   "iat": 1628181264,
   "tmb": "cLj8vsYtMBwYkzoFVZHBZo6SNL8wSdCIjCKAwXNuhOk",
   "typ": "cyphr.me/cyphrpass/account/create",
   "id": "RpMM4_lU6jCj3asZEtIFyYqPjC2L6mlucl7VGMvAuno"
 },
 "sig":
"OigwNSWuPbzkZ6IVaK8MSw5kL61M2cdvuo7HmcfGUJMsRq9I1aD1y1t
MwWFC3BmUu8x_AvFR9BoQ3DTxjNLnpw"
}
```


Since the key may not be known by third parties, it is wise for the user's agent to also send the user's public key along with this transaction:


```
{
 "pay": {
   "alg": "ES256",
   "iat": 1628181264,
   "tmb": "cLj8vsYtMBwYkzoFVZHBZo6SNL8wSdCIjCKAwXNuhOk",
   "typ": "cyphr.me/cyphrpass/account/create",
   "id: "RpMM4_lU6jCj3asZEtIFyYqPjC2L6mlucl7VGMvAuno"
 },
 "key":{
   "alg":"ES256",
   "kid":"Zami's Majuscule Key.",
   "tmb":"cLj8vs...",
   "x":"2nTOaF…"
 }
}
```


Adding Another Key


```
{
"pay": {
 "alg": "ES256",
 "iat": 1628181264,
 "tmb": "cLj8vsYtMBwYkzoFVZHBZo6SNL8wSdCIjCKAwXNuhOk",
 "typ": "cyphr.me/key/upsert",
 "key": {
  "alg": "ES256",
  "iat": 1624472390,
  "tmb": "zxcLp3BEYYoAZxM9QlV7lS4o3Jn1T0dz9L0pWPZJnIs",
  "kid": "Second Key.",
  "x": "F-uHX_..."
  }
}
}
```


Identifiers and Derivations

Also, since Cyphrpass supports multiple cryptographic primitives, for each key alg the associated hashing alg is used to generate identifiers.

An "identity" is abstract, being a digest over a state in a moment of time. The actual concrete digest value for any relevant hashing algorithm is termed a **derivation**. Derivations are identifiers that reference account roots, blocks, auth state, or data state. Although this adds complexity to Cyphrpass, this frees Cyphrpass from tightly coupling to a particular cryptographic primitive. By default, derivations are only calculated for the hashing algorithms referenced by known keys.

Transaction Type Description

Example type: "cyphr.me/ac/image/create"

The first unit is the authority.	(auth)
The last unit is the verb.	(verb)
The second unit is the root.	(root)
Middle units are the noun	(noun)
Trailing nouns are adjectives.	(adj)
Everything after authority is action	(act)
The last noun unit is the child.	(child)

In cases where there is only one noun, that noun is the

noun, root, and child. When a noun has two or more components (such as /ac/image), it is called a compound noun.

Example 1: "cyphr.me/ac/image/create"

Authority: cyphr.me

Verb: create

Root: ac

Noun: ac/image

Action: ac/image/create

Child: image

Example 2: "cyphr.me/image/create"

Authority: cyphr.me

Verb: create

Root: image

Noun: image

Action: image/create

Child: image

The authority itself may be an account root or block ID.

Standard Verbs (CRUD)

All actions can have various fields in addition to `id`

- `create`, `read`, `update` `upsert`, `delete`,`revoke`

Most of these fields should include an `id` field in the payload, although `create` may have it implicitly, or explicitly, since the action itself may generate the `id`. Since the action generates the `id`, the action itself may not be able to refer to it.

`revoke` is special for keys. Has the "rvk" field. See Coz docs on revoke.

Disaster Recovery and Stolen Accounts

There is a danger for services with Cyphrpass's design, since keys may be revoked at a later date, bad actors may gain control of keys used to calculate the original AR. For this reason, AR is **first-to-control**. Most simply, services should keep a ledger of the original state, AR, and current state for all users. This can be kept in a key-value pair.

Paper Wallets

Third Party Verification Services

Social Recovery

Reveal and Public Keys

Account Forking

Time and Oracles

Many oracles can be used as a witness for the legitimate account state.

How does Cyphrpass work?

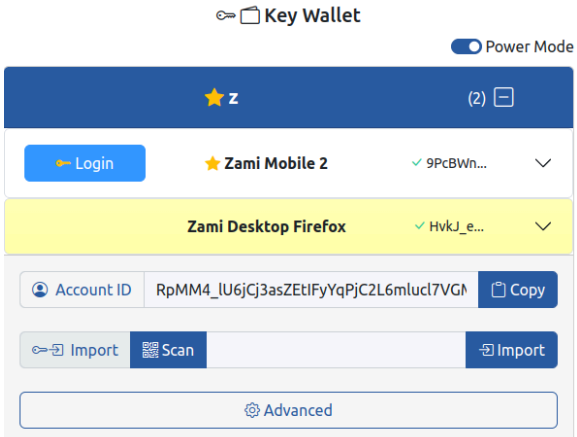
Cyphrpass uses a unique combination of public and private key cryptography to secure your identity. When you create a Cyphrpass account, you generate a public-private key pair. Your public key is shared with the network, while your private key is kept secure on your device. Your private key is used to sign transactions, while your public key is used to verify them.

Whenever you need to verify your identity or share personal information, you sign a transaction with your private key. The network verifies the transaction using your public key and adds it to the blockchain. This creates a tamper-proof record of your identity and any transactions associated with it.

Cyphrpass seeks to liberty, privacy, and security and enabling individuals complete control of online identities

Alternatively to "logging in", each user action is automatically signed. CRUD is accomplished through these actions. (AAA).

Despite the internet revolution significantly expanding human ability, significant challenges in online identity remain. Developments in AI have the potential to upheave many social systems. A robust authentication layer for the Internet is needed more now than ever.



Coze


```
{
 "pay": {
  "alg": "ES256",
  "iat": 1681224485,
  "tmb": "9PcBWntvjAktwfiPp8WxgOyQOwc1h6Lo1UnB_gkWXKk",
  "typ": "cyphr.me/user/login/create"
 },
 "sig": "_enHIu2N_hQawcDozn3v8g_5bCgslvlazeNBx_H6SFci1-aTVm72S5tNyxtjPSHF0WABrbJfyTR7bXxy2HEGAQ"
}
```


Atomic Action (AA)

Currently, user action is hosted by third party services. Users must trust centralized services when

Instead of trusting a central server where authentication is secret from other users, Cyphrpass is designed

Disaster Recovery

Cyphrpass is built with disaster recovery in mind.

Strategies for disaster recover:

1. Paper wallets, cold wallets (offline wallets).
2. Giving a trusted party a backup key.
3. Timelocks on key changes.

Backdated Revokes

Backdated revokes are currently out of scope for Cyphrpass. However, users may sign key revokes and store them in less secure locations. If a device becomes compromised, the user may publish the revoke at a later date.

Suggested API endpoints

GetSupportedAlgs: Lists algorithms that the server supports, e.g. ["ES256","ES384","Ed25519"].

GetSupportedAlgs

GetSupportedSignAlgs

GetSupportedHashAlgs

Traditional Login (Bearer Token)

Cypherpass may also be in place of passwords. After authenticating via proof of possession over TLS, the server may respond with a random string used as a bearer token. As is typical, this requires

Account Forking and Transaction Conflict Resolution

Not all implementations may interpret transactions as intended. One reason for BSL is that conflicts may be easily spotted.

In such cases, changes in transactions that include conflict may be reversed, for example removing a new key that caused the state to conflict. Another possibility is to keep multiple states, which may be managed through forking an account into two. Accounts may be forked by presenting some ledgers

with certain transactions and other ledgers with different transactions. However this paper provides no standard method for account forking. It is our assumption that standard practices will be adopted as Cyphrpass matures and conflict resolution will

Conclusion

FAQ

Is Cyphrpass a blockchain?

No Cyphrpass isn't a blockchain, it's many personal blockchains. Level 3 and up are implemented using personal, non-economic blockchains, however, unlike economic blockchains the only actor that can modify the blockchain is the user. The entirety of a user's blockchain is assumed to be under the total control of a user, and is not designed for economic function.

See Also

- **https://github.com/zamicol/Cyphrpass**
- Bitcoin: A Peer-to-Peer Electronic Cash System
- Trillian and the paper "Verifiable Data Structures"
- VAPID (RFC 8292)
- Ethereum's Account Abstraction

Glossary

Cyphrpass - The authentication layer for the Internet using public key cryptography, updateable keys, and hierarchical digests.

Account Root (AR) - The initial digest used to identify the account. From one or more keys and calculates a Merkle root (MR) from the `tmb` of the keys. When only one key is present, the MR is calculated by taking the digest of the single key. (The MR is the digest of `tmb` so as not to conflict with the key's identifier.) This initial MR is termed the

Auth tree

Action - An action is a cryptographically signed message instructing a service to do something. For example, a "cyphr.me/comment/create" action instructs the website "cyphr.me" to create a comment for the user with specific content. For example:


```
{
  "pay": {
    "msg": "Coze Rocks",
    "alg": "ES256",
    "iat": 1623132000,
    "tmb": "cLj8vsYtMBwYkzoFVZHBZo6SNL8wSdCIjCKAwXNuhOk",
    "typ": "cyphr.me/msg"
  },
  "sig":
"Jl8Kt4nznAf0LGgO5yn_9HkGdY3ulvjg-NyRGzlmJzhncbTkFFn9jrwIwGoRA
QYhjc88wmwFNH5u_rO56USo_w"
}
```


See Coz for more suggested semantics of actions.

Data Ledger State - Non-authentication related actions.

Authentication merkel root -

Authenticated Atomic Action (AAA)- Each action performed by an account is individually signed. These actions may be authenticated by anyone, irrespective of centralized services. For example, instead of logging in to make a comment, a user signs a comment directly which is then verifiable by anyone. Centralized services that maintain user identity, like reddit user accounts, are no longer relevant. See also action.

Account State Resolver (ASR)- Services may query an account state resolver to retrieve the current state of a user. When updating the state of a user, services verify the chains of transactions and update their internally tracked Merkel root for the user. Resolving may be built directly into wallet functionality.

Coz - An open source, human readable cryptographic JSON messaging specification. Coz is a cryptographic abstraction layer enabling cryptographic primitive independence and standardization. See the detailed Coz README and docs for more details on Coz's philosophy and the strategic need.

Derivation - In Cyphrpass, ID's for the account root and block ID's are abstract since they may be calculated with many different algorithms. The digest value for a given hashing algorithm is the derivation.

Mutual State Synchronization (MSS) - Services themselves may be represented by a Cyphrpass Account. User wallets may track service state, and services track user state. This is similar to double entry accounting.

Wallet - An account's storage of keys. For a particular account, many devices may have individual keys.

Crap:

Why use Cyphrpass?

Cyphrpass offers several advantages over traditional identity systems. First, it is decentralized, there is no single point of failure or vulnerability. Second, it is secure and tamper-proof, meaning your identity and personal information cannot be hacked or stolen. Third, it puts you in control of your identity, allowing you to choose what information to share and with whom.

Services state may be updated by publishing signed transactions that update a user's state.

Logout

Log out by updating the global merkle root.

Alternatively, services have to be aware of session tokens.